



testing.md — Orvio Testing Strategy & Engineering Standards

Version: 0.1 (MVP-ready)

Last updated: 2026-09-29

Owner: Engineering (Code X)

Applies to: Frontend, Backend, QA, DevOps, AI coding agents

0. How to Use This Document

This is the **authoritative testing source of truth** for Orvio. It defines what to test, how, where tests belong, and what must pass before code is complete.

Chain of truth:

PRD → architecture.md → database.md → api-contract.md → auth.md → security.md → testing.md → Implementation → Tests → CI/CD → Review

Rules:

- Do not change test strategy without updating this doc.
- Do not remove failing tests to make CI pass.
- AI agents must read this before implementing tests.

1. Source-of-Truth Rule

Hierarchy:

```
PRD.md
↓
architecture.md
↓
database.md
↓
api-contract.md
↓
auth.md
↓
security.md
↓
testing.md
↓
```

```
AGENTS.md
↓
Implementation
↓
Tests
↓
CI/CD
↓
Code Review
```

Conflict resolution: Higher document wins. `testing.md` interprets requirements without changing them.

2. Testing Philosophy

2.1 Principles

- **Test behavior, not implementation.**
- **Deterministic:** Same input → same output.
- **Fast feedback:** Unit/API first, E2E for critical journeys.
- **Critical logic:** Business rules, security boundaries.
- **Failure paths:** Not only happy paths.
- **Edge cases:** Boundaries, concurrency, idempotency.
- **Maintainable:** Simple, clear, minimal mocking.
- **Isolated:** No cross-test dependencies.
- **Reproducible:** Same results locally/CI.

2.2 Testing Pyramid

```
          E2E (critical journeys)
        /
Integration (API + DB, webhooks) \
      /
Unit/Component (domain, validation, UI) \
```

Focus: Strong unit/integration base; minimal, high-value E2E.

3. Testing Objectives

Protect against:

- Functional regressions.
- Business logic regressions.
- API regressions (contract breaks).

- Database regressions (constraints, transactions).
- Auth regressions (login, tokens).
- AuthZ regressions (roles, permissions).
- IDOR/BOLA (cross-tenant access).
- Data corruption, invalid state transitions.
- Race conditions, concurrency issues.
- Validation failures.
- Security vulnerabilities (XSS, injection).
- UI regressions (critical flows).
- Integration failures (payments, WhatsApp).
- Webhook/background job failures.
- Deployment regressions.

4. Test Levels

4.1 Unit Tests

What: Pure functions, domain logic, validation, state transitions, permission logic.

Mock: External services, DB (for pure logic), file system.

Do not mock: Business rules, validation schemas.

Examples:

- Stock calculation (no negative stock).
- Discount limits per role.
- Password hashing/validation.

4.2 Component Tests (Frontend)

What: React components, forms, loading/error/empty states, permission-based UI.

Mock: API calls (MSW), auth state.

Do not mock: Component behavior (render, interactions).

Examples:

- POS cart component (add/remove items).
- Login form (validation, error display).
- Empty state for product list.

4.3 Integration Tests

What: API + DB, service + repository, webhooks, background jobs.

Real: Database (test DB), API layer.

Mock: External services (Paystack, WhatsApp) unless sandbox available.

Examples:

- Create sale → stock reduced.
- Payment webhook → order status updated.
- Stock adjustment → audit log created.

5. API Testing

Source: `api-contract.md`.

For each endpoint category, test:

- **Auth:** Unauthenticated → 401.
- **AuthZ:** Wrong role/org → 403.
- **Validation:** Invalid body → 400.
- **Happy path:** Valid request → 200/201.
- **Not found:** Invalid ID → 404.
- **Conflict:** Duplicate SKU → 409.
- **Forbidden:** Cross-org access → 403.
- **Malformed:** Bad JSON → 400.
- **Boundary:** Max length, negative numbers.
- **Security:** IDOR, tenant isolation.

Endpoints:

- **Auth** (`/auth/*`), **Org** (`/orgs/*`), **Inventory** (`/products`, `/stock/*`), **POS** (`/pos/sales`), **Orders** (`/orders/whatsapp`), **Payments** (`/payments/*`), **Reports** (`/reports/*`).

6. Authentication Testing

Source: `auth.md`.

Test:

- Signup (valid/invalid).
- Login (valid/invalid credentials).
- Logout (token cleared).

- Session creation/expiration.
- Access/refresh token flow.
- Token rotation.
- Password reset (request/confirm).
- Suspended/deleted accounts (login fails).
- Cookie security (httpOnly).
- CSRF (if cookies used).
- CORS behavior.

Distinctions:

- Invalid session ≠ Missing session ≠ Expired session ≠ Revoked session ≠ Auth infrastructure failure.

Regression: Auth failure must not treat infra failure as valid-session invalidation.

7. Authorization Testing

Test:

- AuthN vs AuthZ (separate).
- Role permissions (Owner vs Cashier).
- Resource ownership (user can edit own org only).
- Tenant membership (org_id scope).
- Branch scope (manager vs cashier).
- Admin permissions (internal admin).
- Deny-by-default.

For each protected resource:

- Unauthenticated → 401.
- Authenticated, unauthorized → 403.
- Authorized → 200.
- Authorized, wrong tenant → 403.
- Privileged user → 200 (if allowed).

8. IDOR / BOLA Testing

For every endpoint accepting resource ID:

- Sequential IDs, UUIDs, public-looking IDs.
- URL params, body, query params.

- Nested/related resources.
- Bulk, update, delete, read, export, download.

Test:

- Tenant A user → Tenant B resource → 403.
- User changes ID → still 403/404 (no leakage).

Regression: Any IDOR bug → add authorization regression test.

9. Security Testing

Source: security.md.

Test:

- IDOR/BOLA (see §8).
- Broken access control (role escalation).
- Privilege escalation (cashier → owner).
- Mass assignment (unexpected fields ignored).
- SQL injection (Prisma parameterization).
- XSS (React escaping, CSP).
- CSRF (if cookies).
- SSRF (external URLs validated).
- Path traversal (file uploads).
- Open redirects (validate URLs).
- Security headers (CSP, HSTS).
- CORS misconfiguration.
- Rate-limit bypass.
- Replay attacks (idempotency).
- Webhook forgery (HMAC verification).
- Session/token attacks (expiry, rotation).
- Credential exposure (logs scrubbed).
- Enumeration (generic error messages).

For each:

- Threat, test strategy, expected secure behavior, failure condition, regression test requirement.

10. Database Testing

Source: database.md.

Test:

- Schema integrity (constraints, FKs, uniques).
- Nullability, defaults.
- Index behavior (query plans).
- Transactions (commit/rollback).
- Cascading (FKs).
- Soft delete (deleted_at filter).
- Tenant isolation (org_id scoping).
- Data ownership (user can't access other org).
- State transitions (order status).
- Concurrent writes (race conditions).
- Idempotency (duplicate requests).
- Migrations (correctness, rollback).
- Seed data (fixtures).

Use:

- **Real DB:** For integration tests.
- **Test DB:** Isolated per CI run.
- **Transaction rollback:** For unit tests touching DB.
- **Fixtures/factories:** For test data.

Do not mock: DB for tests verifying DB behavior.

11. Multi-Tenancy Testing

Test:

- Tenant A user → Tenant A resource (allowed).
- Tenant A user → Tenant B resource (denied).
- Tenant admin → Tenant resource (allowed).
- System admin → Tenant resource (allowed if policy).
- Unauthenticated → Tenant resource (denied).

Across:

- Reads, creates, updates, deletes.
- Searches, filters, pagination.

- Exports, reports.
- Files, background jobs, events, webhooks, caches.

Failure: Tenant isolation failure = critical security regression.

12. Business Logic Testing

Source: PRD business rules.

Test:

- State machines (order status transitions).
- Workflow transitions (draft → pending → paid).
- Validation rules (no negative stock).
- Permissions (cashier can't void without reason).
- Limits (discount caps).
- Calculations (subtotal, tax, total).
- Financial logic (NGN minor units).
- Inventory logic (stock movements).
- Notifications (low-stock alerts).
- Audit logging (who did what).

Do not test only UI: Backend/domain layer must protect rules.

13. State Machine Testing

State machines:

- OrderStatus: draft → pending_payment → paid → fulfilled/refunded/cancelled.
- PaymentStatus: pending → succeeded/failed → refunded.

Test:

- Valid transitions.
- Invalid transitions (e.g., draft → fulfilled).
- Repeated transitions (idempotency).
- Skipped transitions (deny).
- Unauthorized transitions (role check).
- Concurrent transitions (race).
- Retry behavior (webhooks).
- Failure recovery (rollback).
- Terminal states (no further transitions).

14. Frontend Testing

Test:

- Rendering (components, pages).
- User interactions (clicks, forms).
- Forms (validation, errors).
- Navigation (routing, protected routes).
- Auth state (login/logout).
- API failures (error toasts).
- Loading states (skeleton/spinner).
- Empty states (CTAs).
- Error states (retry).
- Optimistic updates (POS cart).
- Cache invalidation (TanStack Query).
- Stale data (background refresh).
- Pagination, search, filters.
- Modals/dialogs.
- Toasts/notifications.
- Responsive behavior (mobile POS).
- Accessibility (keyboard, screen reader).

Focus: User-visible behavior, not internal state.

15. Loading / Error / Empty State Testing

For async operations, test:

- Initial → Loading → Success/Error.
- Empty state (no data) → CTA.
- Error state → Retry button.
- Skeleton loading (dashboards, lists).
- Spinner loading (buttons).
- Disabled controls (while loading).
- Optimistic state (rollback on failure).
- Partial loading (infinite scroll).
- Background refresh (stale-while-revalidate).
- Network failure (offline POS).
- Timeout (slow API).

- Unauthorized/Forbidden (redirect).
- Not found (404 page).
- Server error (500 toast).

Ensure:

- Loading indicators don't remain permanently.
- Errors provide recovery path.
- No duplicate requests.

16. E2E Testing

Critical journeys:

1. Signup → Login → Onboarding → Create product → Create sale → Logout.
2. WhatsApp order → Payment link → Webhook → Order fulfilled.
3. Stock adjustment → Audit log → Report.

For each persona:

- Owner: Onboard, invite staff, view reports.
- Cashier: POS sale, end-of-day reconciliation.
- Storekeeper: Stock adjust, transfer.

Failure scenarios:

- Network outage during sale.
- Payment webhook failure.
- Invalid credentials.

Do not E2E-test every tiny interaction.

17. Contract Testing

Source: `api-contract.md`.

Test:

- Request schemas (required/optional fields).
- Response schemas (data types, enums).
- Error formats (code, message, requestId).
- Status codes (200, 201, 400, 401, 403, 404, 409, 422, 429, 500).
- Pagination metadata (nextCursor, hasMore).
- Auth requirements (Bearer token).

Goal: Prevent frontend expects A, backend returns B.

OpenAPI: Future (generate from Fastify schemas).

18. Integration with External Services

Services: Paystack, WhatsApp BSP.

Test:

- **Real integration:** Sandbox mode (if available).
- **Mocked:** MSW/fixtures for CI.
- **Contract:** Payload schemas.
- **Failure:** Timeout, 4xx, 5xx, malformed, rate limit.
- **Retry:** Exponential backoff.
- **Duplicate webhook:** Idempotency.
- **Invalid signature:** Reject.

Do not assume external services always available.

19. Webhook Testing

Test:

- Signature verification (HMAC).
- Auth (endpoint public but signed).
- Event validation (known types).
- Duplicate events (idempotency).
- Replay attacks (old timestamp).
- Out-of-order events (handle gracefully).
- Invalid payloads (400).
- Unknown event types (ignore/log).
- Retry behavior (provider retry).
- Transaction boundaries (atomic updates).

Malicious webhook must not modify unauthorized state.

20. Background Job Testing

Jobs: Webhook processing, scheduled reports (future).

Test:

- Job creation (queue).
- Execution (success/failure).

- Retry (exponential backoff).
- Timeout (kill, retry).
- Duplicate execution (idempotency).
- Dead-letter (after N retries).
- Cancellation (if supported).
- Scheduling (cron).
- Partial failure (some items fail).
- Dependency outage (graceful).

Test: Jobs safe to execute more than once.

21. File Upload / Download Testing

MVP: Minimal (product images).

Test:

- File type validation (MIME).
- File size limits ($\leq 5\text{MB}$).
- Filename handling (sanitize, no path traversal).
- Malicious files (block).
- Unauthorized downloads (403).
- Tenant isolation (org_id).
- Expired signed URLs (403).
- Missing files (404).
- Duplicate files (dedupe).
- Upload failure (rollback).
- Partial upload (error).
- Storage failure (500).

Never trust filenames/MIME.

22. Concurrency / Race Condition Testing

Operations:

- Double sale (same stock).
- Concurrent stock adjustments.
- Token refresh races.
- Webhook races (duplicate events).

- Background job races.

Test:

- Simultaneous requests → consistent state.
- Double submission → idempotent.
- Inventory conflicts → 409 or serialized.
- Token refresh → one wins, other invalidated.

Consistency model: Strong (DB transactions).

23. Idempotency Testing

Operations: Payments, sales, invitations.

Test:

- Request 1 → success.
- Request 1 repeated → same result, no side effects.
- Request 1 retried after timeout → same.
- Request 1 concurrent → one wins.

Verify: No duplicate side effects (stock, payments).

24. Error Handling Testing

Test:

- 400 Bad Request (validation).
- 401 Unauthorized (not authenticated).
- 403 Forbidden (no permission).
- 404 Not Found (resource missing).
- 409 Conflict (duplicate, stock conflict).
- 422 Unprocessable Entity (business rule).
- 429 Rate Limited (too many requests).
- 500 Internal Server Error (server bug).
- 502/503/504 (dependency failures).

Verify:

- Correct status code.
- Correct response structure (`error.code`, `message`, `requestId`).
- Safe error message (no stack traces).
- No sensitive info leakage.

- Correct logging (requestId).
- Frontend handling (toast, retry).
- Retry behavior (429 → backoff).

25. Observability Testing

Test:

- Request IDs (propagated).
- Correlation IDs (logs).
- Structured logs (JSON).
- Audit logs (auth events).
- Security events (login failures).
- Error tracking (Sentry).
- Metrics (latency, error rate).
- Background job failures (logged).
- Auth failures (logged).
- AuthZ failures (logged).
- Dependency failures (logged).

Ensure logs do not expose:

- Passwords, tokens, session secrets, API keys, PII.

26. Performance Testing

When required:

- API latency (p95 < 500ms for MVP).
- DB query performance (slow queries < 100ms).
- Large datasets (pagination, 10k rows).
- Search (full-text, 1k products).
- Bulk operations (future).
- File processing (images).
- Concurrent users (100 concurrent).
- Background jobs (queue depth).

OPEN DECISION: Exact targets (baseline later).

Measure:

- Baseline (current).

- Target (OPEN DECISION).
- Method (k6, Artillery).
- Failure threshold (regression >20%).
- Regression handling (optimize, index).

27. Accessibility Testing

Test:

- Keyboard navigation (tab order).
- Focus management (modals).
- Labels (forms).
- Form errors (aria-invalid).
- Semantic elements (buttons, links).
- Screen-reader compatibility (NVDA/VoiceOver).
- Dialog behavior (focus trap).
- Color-independent meaning (icons + text).
- Accessible loading (aria-busy).
- Accessible error states (aria-live).

Tools: axe-core (automated), manual testing for critical flows.

28. Test Data Strategy

Create:

- **Factories:** `createProduct()`, `createOrder()`.
- **Fixtures:** Seed data for E2E.
- **Builders:** Fluent builders for complex objects.
- **Seed data:** Minimal for CI.
- **Random data:** For fuzzing.
- **Deterministic data:** For reproducible tests.
- **Sensitive data:** Never real PII.
- **Tenant test data:** Org A, Org B.
- **User roles:** Owner, Cashier, Storekeeper.
- **Relationships:** FKs valid.

Cleanup:

- Transaction rollback (unit).

- Truncate tables (integration).
- Never use production credentials/data.

29. Test Database Strategy

Lifecycle:

- **Schema setup:** `prisma migrate`.
- **Migrations:** Applied in CI.
- **Reset:** Truncate before each suite.
- **Transaction:** Wrap unit tests.
- **Parallel:** Isolated DB per worker (if possible).
- **Isolation:** Test DB $\neq$ dev/prod.
- **Seed:** Minimal fixtures.
- **Cleanup:** Post-test truncate.
- **CI:** Ephemeral DB (Docker/service).

Ensure: Tests cannot modify production DB.

30. Mocking Strategy

Prefer real:

- Business logic.
- Validation.
- AuthZ.
- DB (integration).
- API boundaries.

Mock:

- External services (Paystack, WhatsApp).
- File system (for unit).
- Time (for scheduled jobs).
- Email provider (future).

Harmful mocking:

- Everything mocked $\rightarrow$ test proves nothing.
- Mocking business rules.

31. Test Naming and Organization

Conventions:

- **Filenames:** `*.test.ts` (Vitest).
- **Directories:** `apps/api/src/modules/*/tests/`, `apps/web/src/features/*/tests/`.
- **Descriptions:** `describe('POST /products', () => { it('creates a product', ...) })`.
- **Grouping:** By module/feature.
- **Fixtures:** `tests/fixtures/`.
- **Factories:** `tests/factories/`.
- **Helpers:** `tests/helpers/`.
- **Shared utilities:** `packages/shared/tests/`.

Align with repo structure.

32. Test Isolation

Tests must not depend on:

- Execution order.
- Another test's DB state.
- Developer's machine.
- Local credentials.
- Previous test results.
- External services being available.
- Wall-clock timing (use fake timers).

Enforce:

- Transaction rollback.
- Truncate between tests.
- Fake timers for time-dependent tests.

33. Flaky Test Policy

Flaky test: Passes/fails without code change.

Detection: CI retries, flaky detection tool.

Investigation:

1. Reproduce locally.
2. Check timing, external deps.

3. Fix root cause (race, timeout).

Quarantine: Mark as skip with ticket link.

Ownership: Assign to engineer.

Fix expectation: Within 1 sprint.

CI behavior: Flaky tests fail build (unless quarantined).

34. Test Coverage

Interpretation: Coverage $\neq$ quality.

Cover:

- Line coverage.
- Branch coverage.
- Function coverage.
- Critical-path coverage (auth, authZ, business rules).
- Security-path coverage (IDOR, tenant isolation).

OPEN DECISION: Thresholds (recommend 80% critical paths).

Critical paths: Stronger coverage than utility code.

35. Regression Testing

Rule:

```
Bug discovered
↓
Root cause identified
↓
Fix implemented
↓
Regression test added
↓
Relevant suite executed
```

Prevent: Same bug silently returning.

36. Security Regression Testing

Examples:

- IDOR discovered $\rightarrow$ authorization regression test.
- Privilege escalation $\rightarrow$ permission regression test.

- Tenant isolation issue → cross-tenant access test.
- Session bug → session lifecycle regression test.
- Webhook forgery → signature verification regression test.

37. CI/CD Testing

Stages:

```
Local
↓
Lint / Typecheck
↓
Unit Tests
↓
Component Tests
↓
Integration Tests
↓
API Tests
↓
Security Tests
↓
E2E (critical)
↓
Build
↓
Deploy Staging
↓
Smoke Tests
↓
Deploy Prod
```

PR checks:

- Lint, typecheck, unit, integration, API, security.

Main branch:

- All above + E2E.

Pre-deployment:

- Smoke tests (auth, critical API).

Post-deployment:

- Health checks, critical journey.

Failure handling:

- Block merge/deploy on critical failures.

38. Local Development Testing

Fast feedback:

- Targeted tests (changed module).
- Full suite before merge.
- DB tests (local Postgres).
- API tests (local Fastify).
- Frontend tests (Vitest + RTL).
- E2E (Playwright) for critical flows.
- Typecheck, lint.
- Security checks (audit, SAST if available).

39. Testing AI-Generated Code

AI agents must:

- Inspect existing tests before creating new.
- Reuse existing infrastructure (fixtures, factories).
- Avoid duplicate utilities.
- Avoid inventing frameworks.
- Never remove failing tests to make CI pass.
- Never weaken assertions.
- Never disable security tests.
- Add regression tests for bugs.
- Test authZ boundaries, tenant isolation.
- Test failure paths, edge cases.
- Run smallest relevant suite first.
- Run broader suite before completion.
- Report tests not executed (NOT RUN, reason).
- Never fabricate results.

Understand: Passing tests $\neq$ correct if tests are incomplete.

40. Testing Change Impact Analysis

For every change:

```
Changed code
  ↓
Affected module
```

↓
Affected business rules
↓
Affected API
↓
Affected DB behavior
↓
Affected auth/authZ
↓
Affected security boundaries
↓
Affected UI
↓
Affected integrations
↓
Required tests

AI agents: Perform impact analysis before large changes.

41. Test Matrix

Area	Test Type	Risk	Required	Owner	Trigger
Authentication	Integration	High	Yes	Backend	Auth changes
Authorization	Security	Critical	Yes	Backend	Permission changes
Business Logic	Unit	High	Yes	Engineering	Logic changes
API	Contract	High	Yes	Backend	API changes
UI	Component	Medium	As applicable	Frontend	UI changes
Critical Journey	E2E	High	Yes	QA/Engineering	Major workflow changes
Database	Integration	High	Yes	Backend	Schema/data changes
Multi-Tenancy	Security	Critical	Yes	Backend	Tenant changes
Webhooks	Integration	High	Yes	Backend	Webhook changes
Background Jobs	Integration	Medium	Yes	Backend	Job changes

42. Requirement Traceability

Map PRD requirements to tests:

Requirement (PRD-001)
↓
Business Rule (no negative stock)
↓
Implementation (stock adjustment service)
↓
Test (unit: adjust stock → reject if negative)



Acceptance Criteria (AC-INV-003)

Use actual PRD IDs.

43. Acceptance Testing

Structure:

```
Given: Initial condition
When: User/system action
Then: Expected result
```

Cover:

- Happy path.
- Failure path.
- AuthZ.
- Validation.
- Edge cases.
- Security constraints.

44. Testing Checklist for New Features

- ☐ Unit tests.
- ☐ Component tests.
- ☐ Integration tests.
- ☐ API tests.
- ☐ Database tests.
- ☐ Authentication tests.
- ☐ Authorization tests.
- ☐ IDOR/BOLA tests.
- ☐ Tenant isolation tests.
- ☐ Validation tests.
- ☐ Error tests.
- ☐ Loading-state tests.
- ☐ Empty-state tests.
- ☐ E2E tests (critical journey).
- ☐ Security regression tests.

- [] Performance tests (if applicable).
- [] Accessibility tests (if UI).
- [] Regression tests.
- [] Documentation updated.

45. Bug Report Requirements

Minimum:

- Environment (local/staging/prod).
- Version/commit.
- Preconditions (seed data).
- User role (Owner, Cashier).
- Tenant/org context.
- Steps to reproduce.
- Expected behavior.
- Actual behavior.
- Error info (message, requestId).
- Logs (safe, scrubbed).
- Screenshots/videos.
- Severity (critical/high/medium/low).
- Security impact (yes/no).
- Reproduction frequency (always/sometimes).

46. Test Failure Investigation

Workflow:

```
Test fails
  ↓
Reproduce locally
  ↓
Determine whether test or implementation is wrong
  ↓
Inspect logs/errors
  ↓
Identify root cause
  ↓
Fix root cause
  ↓
Run focused test
  ↓
Run related suite
```

↓
Run broader suite
↓
Add regression coverage if necessary

Prohibit: Blindly changing assertions until test passes.

47. Test Severity / Priority

Severity scale (RECOMMENDATION):

- **Critical:** Security, data loss, tenant isolation, auth failure.
- **High:** Critical workflow broken, API regression.
- **Medium:** UI regression, non-critical bug.
- **Low:** Cosmetic, minor UX.

Priority: Critical → High → Medium → Low.

48. Production Validation

Post-deploy:

- Health checks (/health).
- Smoke tests (login, create product, create sale).
- Critical API endpoints (GET /products, POST /pos/sales).
- Critical user journey (sale flow).
- DB connectivity.
- External integrations (Paystack webhook).
- Background jobs (webhook processor).
- Error monitoring (Sentry).
- Metrics (latency, error rate).
- Rollback verification (if deployed).

Do not introduce destructive production tests.

49. Disaster / Failure Scenario Testing

Test:

- Database outage (500, graceful).
- API dependency outage (fallback).
- Auth dependency outage (deny access).
- Cache outage (degraded).

- Queue outage (job retry).
- Storage outage (upload fail).
- Third-party outage (Paystack down → queue).
- Network timeout (retry).
- Partial deployment (canary).
- Background job failure (dead-letter).

Verify: Graceful degradation, safe failure.

50. Testing Invariants

Must always remain true:

- Users cannot access resources outside authorized scope.
- Users cannot access another tenant's data.
- Unauthorized users cannot invoke privileged operations.
- Invalid state transitions are rejected.
- Sensitive credentials never appear in logs.
- Duplicate idempotent requests do not duplicate side effects.
- Authentication infrastructure failures are not interpreted as valid-session invalidation.
- Deleted/suspended accounts cannot continue unauthorized access.
- Security controls cannot be bypassed by manipulating resource IDs.

51. What Not to Test

- Framework internals (React, Fastify).
- Third-party library internals (Prisma, Zod).
- Trivial implementation details (getters/setters).
- Generated code (unless critical).
- Static values with no behavior.
- Duplicate tests covering same behavior.

Goal: High confidence, not max test count.

52. Test Maintenance

Evolve:

- Remove obsolete tests.
- Update tests after intentional behavior changes.

- Refactor test utilities.
- Keep fixtures realistic.
- Prevent duplicated helpers.
- Review flaky tests (fix or quarantine).
- Review slow tests (optimize).
- Review unnecessary mocks (replace with real).

Tests are production code.

53. Test Performance

Monitor:

- Slow tests (>1s unit, >5s integration).
- Slow suites (>10min CI).
- DB-heavy tests (optimize queries).
- E2E-heavy tests (parallelize).
- Setup overhead (shared fixtures).
- Parallelization opportunities (workers).

Do not prematurely optimize at expense of reliability.

54. Test Security

Secure:

- Test credentials (never commit).
- Secrets (env vars, masked in CI).
- Test databases (isolated).
- Production-data restrictions (never use real PII).
- Logs (scrubbed).
- Screenshots (no sensitive data).
- Test artifacts (temporary files deleted).
- Mock credentials (fake, not real).
- External test services (sandbox).

Never commit real credentials.

55. Testing Dependencies

Rules:

- Reuse existing deps (Vitest, RTL, Playwright).
- Avoid adding new framework for trivial test.
- Avoid duplicate assertion/mocking libraries.
- Justify new testing deps (security, performance).
- Keep compatible with project architecture.

56. Open Decisions

- **Testing framework:** Vitest (frontend/backend) — CONFIRMED (assumed from stack).
- **Coverage thresholds:** OPEN DECISION (recommend 80% critical paths).
- **CI provider:** GitHub Actions — RECOMMENDATION.
- **Browser matrix:** Chrome, Firefox — OPEN DECISION.
- **Device matrix:** Mobile (Android), Desktop — OPEN DECISION.
- **Load-testing tool:** k6 — RECOMMENDATION.
- **Accessibility tooling:** axe-core — RECOMMENDATION.
- **Security scanning:** Snyk, OWASP ZAP — RECOMMENDATION.
- **Test environment architecture:** Ephemeral DB per CI run — RECOMMENDATION.

57. Recommendations

- Adopt Vitest for unit/component (fast, TS-native).
- Playwright for E2E (reliable, cross-browser).
- MSW for API mocking (frontend).
- Testcontainers for integration (Postgres in CI).
- k6 for performance (API latency).
- axe-core for accessibility (automated).
- Snyk for dependency scanning.

58. Testing Risks

Risk	Mitigation
Insufficient coverage	Focus on critical paths, business rules.
Over-reliance on E2E	Strong unit/integration base.
Excessive mocking	Use real DB/logic for integration.

Risk	Mitigation
Missing authZ tests	Mandatory authZ test per endpoint.
Missing tenant-isolation tests	Cross-tenant test per resource.
Flaky tests	Quarantine, fix root cause.
Slow CI	Parallelize, optimize slow tests.
Missing integration tests	Test API+DB, webhooks, jobs.
Missing regression tests	Add regression for every bug.
Environment differences	Use containers, ephemeral DB.
Unstable external deps	Mock in CI, sandbox for integration.

59. Implementation Order

1. Test infrastructure: Vitest, Playwright, MSW.
2. Test database: Ephemeral Postgres (Testcontainers).
3. Unit/domain tests: Business logic, validation.
4. API tests: Contract, auth, authZ.
5. Auth tests: Login, tokens, reset.
6. AuthZ/security tests: IDOR, tenant isolation.
7. Integration tests: API+DB, webhooks, jobs.
8. Frontend/component tests: Forms, loading, error.
9. Critical E2E flows: Signup → sale.
10. CI integration: GitHub Actions.
11. Performance/security testing: k6, Snyk.
12. Production smoke testing: Post-deploy.

60. Final Testing Reference

60.1 Testing Principles

- Test behavior, not implementation.
- Deterministic, fast feedback.
- Critical logic, security boundaries.
- Failure paths, edge cases.
- Maintainable, isolated, reproducible.

60.2 Test Levels

- **Unit:** Domain, validation.
- **Component:** UI, forms.
- **Integration:** API+DB, webhooks.
- **E2E:** Critical journeys.

60.3 Critical Security Tests

- IDOR/BOLA.
- Tenant isolation.
- AuthZ (role, permission).
- Webhook signature.
- Token expiry/rotation.

60.4 Critical Business Tests

- No negative stock.
- Order state transitions.
- Payment idempotency.
- Discount limits.

60.5 Required Regression Rules

- Every bug → regression test.
- Security fix → security regression.
- API change → contract test update.

60.6 AI Coding Agent Rules

- Read `testing.md`, `AGENTS.md`.
- Reuse existing tests/fixtures.
- Never remove failing tests.
- Test authZ, tenant isolation.
- Run tests before completion.

60.7 Pre-Merge Checklist

- [] Lint, typecheck.
- [] Unit, integration tests pass.
- [] API tests pass.
- [] Security tests pass.

- ☐ No new vulnerabilities.
- ☐ Docs updated.

60.8 Pre-Deployment Checklist

- ☐ All tests pass.
- ☐ Smoke tests pass.
- ☐ Secrets configured.
- ☐ Migrations applied.
- ☐ Rollback plan ready.

60.9 Post-Deployment Checklist

- ☐ Health checks pass.
- ☐ Critical journey works.
- ☐ Error monitoring clear.
- ☐ Metrics normal.
- ☐ Rollback verified (if needed).

End of testing.md